
Manual de Referencia de Python Core

Consulta completa, clara y estructurada para repasar, resolver dudas y leer código con criterio

Este manual está pensado para alguien que **ya programa en Python**, ha resuelto ejercicios y proyectos, y quiere tener un documento único para:

- repasar conceptos sin volver a empezar desde cero;
- entender líneas de código que a primera vista parecen raras;
- consultar sintaxis, reglas y decisiones de diseño;
- leer proyectos más grandes con seguridad;
- volver a la documentación oficial con mejor contexto.

No es un curso paso a paso para principiantes absolutos. Es una **referencia explicada**: conserva la precisión técnica, pero traduce las partes más densas a una intuición más amigable.

Criterio editorial: si un término, símbolo o forma de escribir código suele generar dudas, aquí se explica. Si una práctica parece “más formal” de lo necesario, se aclara cuándo vale la pena y cuándo no.

Cómo usar este manual

Puedes leerlo de principio a fin una vez, pero su valor real está en usarlo como mapa de consulta.

Rutas de lectura recomendadas

Necesidad	Secciones sugeridas
Repasar sintaxis y lógica general	1 a 11
Entender listas, diccionarios y mutabilidad	12 a 16
Leer mejor funciones, módulos y proyectos	17 a 22
Entender POO, tipos y diseño	23 a 25
Trabajar con librerías estándar y archivos	21, 26 y 27
Mejorar calidad de proyectos	28 a 31
Aclarar vocabulario técnico	32

Contenido general

1. Modelo mental de Python: objetos, nombres y referencias
2. Tipos básicos, conversiones y verdad lógica
3. `None`, ausencia de valor y valores centinela
4. Entrada, salida y `print()` con `sep`, `end` y escapes
5. Operadores, matemáticas, división y precisión numérica
6. Condicionales, comparaciones, pertenencia, identidad y `match`
7. Cadenas, Unicode, bytes, codificación y limpieza de texto
8. Índices, negativos, slicing, `:` y `range()` explicado con calma
9. Formato de texto: f-strings, `.format()` y especificadores

- 10. Bucles, iteración y control de flujo
- 11. Desempaquetado y asignaciones útiles
- 12. Colecciones: listas, tuplas, conjuntos y `range`
- 13. Listas por dentro: mutación, inserciones y costo conceptual
- 14. Diccionarios y mapeos
- 15. Mutabilidad, aliasing y copias
- 16. Comprensiones, iterables, iteradores y generadores
- 17. Funciones, parámetros, valores por defecto y `return`
- 18. Scope, namespaces, LEGB, `global` y `nonlocal`
- 19. Recursión, funciones anidadas y cierres básicos
- 20. Módulos, imports, paquetes y nombres de archivo
- 21. `main()`, `__name__` == `"__main__"` y estructura de programas
- 22. Archivos, rutas, `__file__`, `os.path` y `pathlib`
- 23. Excepciones, validación de input y errores controlados
- 24. Programación orientada a objetos
- 25. Anotaciones de tipo, `typing` y `dataclasses`
- 26. Biblioteca estándar de consulta frecuente
- 27. Expresiones regulares y strings crudos
- 28. Depuración, pruebas y logging
- 29. Patrones de proyectos: menús, archivos de texto y reportes
- 30. Eficiencia, búsqueda y ordenamiento
- 31. Buenas prácticas y trampas frecuentes
- 32. Glosario técnico
- 33. Fuentes oficiales consultadas

1. Modelo mental de Python: objetos, nombres y referencias

Python se entiende mucho mejor cuando se piensa así:

Un programa no manipula “variables como cajas rígidas”; manipula **objetos** y **nombres que apuntan a esos objetos**.

Conceptos básicos

Concepto	Explicación precisa	Intuición sencilla
Objeto	Entidad con identidad, tipo y valor.	Algo que existe en memoria.
Nombre	Identificador usado para referirse a un objeto.	La etiqueta que tú escribes.
Asignación	Vincula un nombre con un objeto.	“A partir de ahora, este nombre apunta a esto”.

Concepto	Explicación precisa	Intuición sencilla
Tipo	Define operaciones y comportamiento del objeto.	Qué clase de cosa es.
Identidad	Distingue un objeto concreto de otro.	Si son literalmente el mismo objeto.
Referencia	Vínculo entre un nombre y un objeto.	Una flecha del nombre hacia el objeto.

```
lista = [1, 2, 3]
otra = lista
```

Aquí no se crea automáticamente una segunda lista independiente. `otra` y `lista` apuntan al mismo objeto.

```
otra.append(4)
print(lista) # [1, 2, 3, 4]
```

Asignar no es copiar

Esta regla evita muchos errores:

```
a = [10, 20]
b = a
```

- `a` y `b` son nombres distintos.
- Ambos apuntan a la **misma lista**.
- Si la lista cambia por `b`, se ve por `a`.

Tipado dinámico

Python es dinámicamente tipado. El tipo pertenece al objeto, no al nombre.

```
x = 10 # x apunta a un int
x = "diez" # ahora x apunta a un str
```

Esto **no** significa que Python “mezcle todo sin control”. Python sí protege operaciones incompatibles:

```
"5" + 2 # TypeError
```

Precisión: Python es dinámicamente tipado, pero no débilmente tipado en el sentido cotidiano de “acepta cualquier mezcla de tipos”.

2. Tipos básicos, conversiones y verdad lógica

Tipos integrados fundamentales

Tipo	Ejemplo	Idea central
<code>int</code>	<code>42</code>	Entero de precisión arbitraria.
<code>float</code>	<code>3.14</code>	Punto flotante; aproxima muchos decimales.
<code>complex</code>	<code>2 + 3j</code>	Número complejo.
<code>bool</code>	<code>True</code> , <code>False</code>	Valores lógicos.
<code>str</code>	<code>"texto"</code>	Texto Unicode inmutable.
<code>NoneType</code>	<code>None</code>	Ausencia intencional de valor.
<code>bytes</code>	<code>b"abc"</code>	Secuencia inmutable de bytes.
<code>bytearray</code>	<code>bytearray(b"abc")</code>	Bytes mutables.

Conversiones comunes

Función	Qué hace	Nota útil
<code>int(x)</code>	Convierte a entero cuando es posible.	Con floats, trunca hacia cero.
<code>float(x)</code>	Convierte a decimal de punto flotante.	Puede aproximar valores.
<code>str(x)</code>	Produce texto legible.	No siempre igual a <code>repr(x)</code> .
<code>bool(x)</code>	Evalúa verdad lógica.	Colecciones vacías suelen ser <code>False</code> .
<code>type(x)</code>	Muestra tipo exacto.	Útil para inspeccionar.
<code>isinstance(x, T)</code>	Verifica pertenencia a tipo o jerarquía.	Suele ser más flexible que <code>type(x) == T</code> .

Truncar no es redondear

```
int(3.9)    # 3
int(-3.9)   # -3
```

Truncar es **cortar la parte decimal**. No busca el entero más cercano.

Verdad lógica

Se evalúan como `False` de forma típica:

Grupo	Ejemplos
Ausencia	<code>None</code>
Falso lógico	<code>False</code>

Grupo	Ejemplos
Ceros	<code>0, 0.0, 0j</code>
Vacíos	<code>"", [], {}, set(), (), range(0)</code>

Todo lo demás suele ser `True`, salvo que una clase defina reglas especiales.

```
if lista:
    print("Tiene elementos")
else:
    print("Está vacía")
```

3. `None`, ausencia de valor y valores centinela

`None` significa:

“No hay un valor útil aquí”, “no se suministró”, o “todavía no existe resultado”.

No es lo mismo que:

- `0`;
- `False`;
- `""`;
- `[]`.

Ejemplos

```
resultado = None
```

Esto expresa que la variable existe, pero no contiene un resultado calculado.

```
if resultado is None:
    print("Aún no se ha calculado")
```

Comparar con `None`

La forma recomendada es:

```
x is None
x is not None
```

No se acostumbra usar:

```
x == None
```

porque `None` representa una identidad única y `is` comprueba identidad.

None como valor por defecto

Una firma como esta es muy común:

```
def imprimir_tabla(tabla, ancho, encabezado=None):  
    ...
```

Significa:

- `tabla` y `ancho` son obligatorios;
- `encabezado` es opcional;
- si no lo entregan, su valor será `None`.

Luego dentro de la función se puede escribir:

```
if encabezado is not None:  
    # imprimir encabezado
```

Por qué no usar una lista vacía como valor por defecto

Evita esto:

```
def agregar(valor, lista=[]):  
    lista.append(valor)  
    return lista
```

El problema es que esa lista se crea **una sola vez**, cuando se define la función, y se reutiliza entre llamadas.

Prefiere:

```
def agregar(valor, lista=None):  
    if lista is None:  
        lista = []  
    lista.append(valor)  
    return lista
```

Regla práctica: si un valor por defecto debería crear un objeto nuevo en cada llamada, usa `None` y construye el objeto dentro.

4. Entrada, salida y `print()` con `sep`, `end` y escapes

`input()`

```
nombre = input("Nombre: ")
```

`input()` siempre retorna `str`, aunque el usuario escriba números.

```
edad = int(input("Edad: "))
```

Si el texto no puede convertirse, ocurre `ValueError`.

`print()`

Forma general:

```
print(*objetos, sep=" ", end="\n")
```

Parámetro	Qué controla
<code>*objetos</code>	Lo que se imprime.
<code>sep</code>	Separador entre objetos.
<code>end</code>	Lo que se imprime al final.

`sep`

```
print("A", "B", "C", sep="- ")  
# A-B-C
```

`end`

Por defecto, `print()` termina con salto de línea.

```
print("Hola")  
print("Mundo")
```

produce:

```
Hola  
Mundo
```

Con `end=""`:

```
print("Hola", end="")  
print("Mundo")
```

produce:

```
HolaMundo
```

Con `end=" "`:

```
print("Hola", end=" ")  
print("Mundo")
```

produce:

Hola Mundo

Escapes frecuentes

Escape	Significado
<code>\n</code>	Salto de línea.
<code>\t</code>	Tabulación.
<code>\\</code>	Barra invertida literal.
<code>\"</code>	Comilla doble dentro de string con comillas dobles.
<code>\'</code>	Comilla simple dentro de string con comillas simples.

```
print("Hola\nMundo")
```

Limpiar saltos y tabulaciones dentro de texto

Esta línea:

```
texto = texto.replace("\n", " ").replace("\t", " ")
```

se lee así:

1. reemplaza cada salto de línea por un espacio;
2. sobre el resultado, reemplaza cada tabulación por un espacio.

Es una cadena de métodos. Como `str` es inmutable, cada `replace()` retorna un texto nuevo.

Espacios calculados para alinear impresión

```
print(" " * cantidad, end="")
```

- `" " * cantidad` crea una cadena con `cantidad` espacios;
- `end=""` evita saltar de línea.

Ejemplo:

```
print("Hola", end="")  
print(" " * 4, end="")  
print("Mundo")
```

Salida:

```
Hola    Mundo
```

5. Operadores, matemáticas, división y precisión numérica

Operadores aritméticos

Operador	Ejemplo	Resultado conceptual
<code>+</code>	<code>a + b</code>	Suma o concatenación compatible.
<code>-</code>	<code>a - b</code>	Resta.
<code>*</code>	<code>a * b</code>	Producto o repetición de secuencias.
<code>/</code>	<code>a / b</code>	División real, produce <code>float</code> en enteros.
<code>//</code>	<code>a // b</code>	División piso.
<code>%</code>	<code>a % b</code>	Residuo coherente con <code>//</code> .
<code>**</code>	<code>a ** b</code>	Potencia.

División piso no equivale siempre a truncar

```
-3 // 2 # -2  
int(-3 / 2) # -1
```

- `//` aplica piso matemático.
- `int()` sobre un `float` trunca hacia cero.

Módulo `%`

```
17 % 5 # 2
```

El residuo se usa para:

- saber si un número es par: `n % 2 == 0`;
- extraer dígitos;
- ciclos periódicos;
- calcular sobrantes.

Floats y precisión

```
0.1 + 0.2 == 0.3 # False en muchas situaciones
```

Los floats almacenan aproximaciones binarias. Para comparar valores calculados, a menudo conviene usar tolerancias.

```
abs((0.1 + 0.2) - 0.3) < 1e-9
```

Funciones matemáticas comunes

Función	Uso
<code>round(x, n)</code>	Muestra un valor redondeado a <code>n</code> decimales.
<code>abs(x)</code>	Valor absoluto.
<code>max(...)</code>	Máximo.
<code>min(...)</code>	Mínimo.
<code>sum(iterable)</code>	Suma.
<code>divmod(a, b)</code>	Retorna cociente y residuo.

Módulo `math`

Elemento	Idea
<code>math.pi</code>	Constante pi.
<code>math.sqrt(x)</code>	Raíz cuadrada.
<code>math.ceil(x)</code>	Techo.
<code>math.floor(x)</code>	Piso.
<code>math.trunc(x)</code>	Truncamiento hacia cero.

6. Condicionales, comparaciones, pertenencia, identidad y `match`

`if, elif, else`

```
if nota >= 3:
    print("Aprobó")
elif nota >= 2:
    print("Habilita")
else:
    print("Reprobó")
```

Comparaciones

Operador	Significado
<code>==</code>	Igualdad de valor.
<code>!=</code>	Diferencia.
<code><, >, <=, >=</code>	Orden.

Operadores lógicos

Operador	Uso
<code>and</code>	Ambas condiciones deben cumplirse.
<code>or</code>	Basta una condición verdadera.
<code>not</code>	Niega una condición.

Pertenencia

```
letra in "aeiou"
```

Significa: “¿esta letra pertenece a esa colección?”.

En diccionarios:

```
"clave" in diccionario
```

prueba claves, no valores.

Identidad: `is`

```
x is None
```

`is` pregunta si dos nombres apuntan al mismo objeto, no solo si sus valores se ven iguales.

Cortocircuito

Python puede dejar de evaluar condiciones cuando ya conoce el resultado.

```
if lista and lista[0] > 0:  
    ...
```

Si `lista` está vacía, no intenta acceder a `lista[0]`; eso evita `IndexError`.

Expresión condicional

```
estado = "aprobado" if nota >= 3 else "reprobado"
```

Úsala para decisiones breves. Si se vuelve difícil de leer, usa `if` normal.

`match / case`

Desde Python 3.10 existe `match`, útil para patrones estructurados.

```

match opcion:
    case "1":
        print("Crear")
    case "2":
        print("Editar")
    case _:
        print("Opción inválida")

```

No reemplaza todos los `if`. Es útil cuando comparas una misma expresión contra varios patrones.

7. Cadenas, Unicode, bytes, codificación y limpieza de texto

`str` es texto Unicode

Una cadena `str` no es una secuencia de bytes crudos. Representa texto.

Término	Explicación
Unicode	Estándar de caracteres de muchos sistemas de escritura.
Code point	Número abstracto asignado a un carácter.
Glifo	Forma visual que dibuja una fuente.
Codificación	Conversión entre texto y bytes.
UTF-8	Codificación muy usada.

`str` VS `bytes`

```

texto = "á"
bytes_utf8 = texto.encode("utf-8")
texto_de_vuelta = bytes_utf8.decode("utf-8")

```

Métodos de strings

Método	Uso
<code>s.lower()</code>	Minúsculas.
<code>s.upper()</code>	Mayúsculas.
<code>s.strip()</code>	Quita bordes.
<code>s.split()</code>	Divide.
<code>sep.join(lista)</code>	Une strings.
<code>s.replace(a, b)</code>	Sustituye.
<code>s.find(sub)</code>	Índice o <code>-1</code> .

Método	Uso
<code>s.count(sub)</code>	Cuenta ocurrencias.
<code>s.startswith(pref)</code>	Verifica inicio.
<code>s.endswith(suf)</code>	Verifica final.
<code>s.isdigit()</code>	Comprueba dígitos Unicode.
<code>s.isalpha()</code>	Comprueba letras Unicode.

`strip()` no limpia todo

```
"  Hola mundo  ".strip() # "Hola mundo"
```

No borra espacios internos.

`split()` con y sin separador

```
"a b c".split() # ['a', 'b', 'c']
"a,b,c".split(",") # ['a', 'b', 'c']
```

Si no das separador, Python separa por bloques de espacios en blanco.

Strings inmutables

```
texto = "hola"
texto.upper()
print(texto) # sigue siendo 'hola'
```

Debes guardar el retorno:

```
texto = texto.upper()
```

8. Índices, negativos, slicing, `:` y `range()` explicado con calma

Esta sección resuelve una de las dudas más frecuentes: **cuándo aparece `-1`, para qué sirven los `:`, y por qué los rangos parecen terminar “antes”**.

Índices normales y negativos

```
palabra = "Python"
```

Carácter	P	y	t	h	o	n
Índice normal	0	1	2	3	4	5
Índice negativo	-6	-5	-4	-3	-2	-1

```
palabra[0] # 'P'
palabra[-1] # 'n'
palabra[-2] # 'o'
```

La regla del slicing

Forma general:

```
secuencia[inicio:fin:paso]
```

Parte	Regla
inicio	Se incluye.
fin	No se incluye.
paso	Cada cuánto avanza. Puede ser negativo.

Ejemplos básicos

```
palabra[1:4] # 'yth'
```

Toma posiciones 1, 2 y 3. **No toma la 4.**

```
palabra[:3] # 'Pyt'
palabra[3:] # 'hon'
```

¿Qué significa `[:-1]`?

```
palabra[:-1]
```

Se lee:

“Desde el inicio hasta antes del último carácter”.

Resultado:

```
Pytho
```

Por eso esta línea:

```
linea = linea[:-1] + "+"
```

significa:

1. toma la cadena `linea` sin su último carácter;
2. añade `+` al final.

Sirve para reemplazar el último símbolo por otro.

¿Qué significa `[::-1]`?

```
palabra[::-1] # 'nohtyP'
```

- inicio omitido;
- fin omitido;
- paso `-1`.

Se recorre toda la secuencia hacia atrás.

`range()` sigue la misma idea de “fin exclusivo”

```
range(inicio, fin, paso)
```

Ejemplos:

```
list(range(5))          # [0, 1, 2, 3, 4]
list(range(2, 6))       # [2, 3, 4, 5]
list(range(5, 0, -1))   # [5, 4, 3, 2, 1]
list(range(5, -1, -1)) # [5, 4, 3, 2, 1, 0]
```

Regla mental útil

Forma	Cómo pensarla
<code>lista[a:b]</code>	Desde <code>a</code> hasta antes de <code>b</code> .
<code>range(a, b)</code>	Desde <code>a</code> hasta antes de <code>b</code> .
<code>-1</code> como índice	Último elemento.
<code>-1</code> como paso	Avanzar hacia atrás.

`len(lista) - 1`

Si una lista tiene 5 elementos, sus posiciones van de 0 a 4.

```
ultima_posicion = len(lista) - 1
```

Esto aparece mucho cuando trabajas por índices.

9. Formato de texto: f-strings, `.format()` y especificadores

F-strings

```
nombre = "Ana"
nota = 4.756
print(f"{nombre} obtuvo {nota:.2f}")
```

Salida:

```
Ana obtuvo 4.76
```

`.format()`

```
"{} obtuvo {:.2f}".format(nombre, nota)
```

Hace el mismo tipo de inserción.

Especificadores frecuentes

Especificador	Efecto
<code>:.2f</code>	Dos decimales.
<code>:8</code>	Ancho mínimo 8.
<code>:<10</code>	Alinear a la izquierda.
<code>:>10</code>	Alinear a la derecha.
<code>:^10</code>	Centrar.
<code>:08</code>	Rellenar con ceros.
<code>:,</code>	Separador de miles con coma.
<code>:_</code>	Separador de miles con guion bajo.

Desarmando `"{: ^7.2f}".format(texto)`

Supón que `texto` es `3.5`.

```
texto = "{: ^7.2f}".format(texto)
```

La parte `: ^7.2f` se lee así:

Símbolo	Significado
<code>:</code>	A partir de aquí vienen instrucciones de formato.
<code>^</code>	Centrar el contenido.

Símbolo	Significado
7	Ocupar 7 caracteres de ancho total.
.2f	Mostrar como decimal con 2 cifras.

Resultado aproximado:

```
" 3.50 "
```

Sirve mucho para tablas alineadas.

Alinear celdas manualmente

Si una celda tiene espacio para 10 caracteres y tu texto usa 6, faltan 4 espacios. Por eso aparecen fórmulas como:

```
print(" " * (relleno - (len(texto) - impresos)), end="")
```

La idea es:

- calcular cuántos caracteres faltan para completar la celda;
- imprimir esa cantidad de espacios;
- no saltar de línea todavía.

10. Bucles, iteración y control de flujo

while

Úsalo cuando no sabes de antemano cuántas veces repetirás.

```
while True:
    respuesta = input("s/n: ")
    if respuesta in ("s", "n"):
        break
```

for

Úsalo cuando recorres elementos.

```
for nombre in nombres:
    print(nombre)
```

range()

```
for i in range(5):
    print(i)
```

Imprime 0, 1, 2, 3, 4.

enumerate()

```
for posicion, nombre in enumerate(nombres, start=1):  
    print(posicion, nombre)
```

Evita crear contadores manuales cuando el índice importa.

zip()

```
for nombre, nota in zip(nombres, notas):  
    print(nombre, nota)
```

Agrupar por posición.

break, continue y else

Sentencia	Efecto
break	Salir del bucle actual.
continue	Salta a la siguiente iteración.
else en bucle	Se ejecuta si el bucle terminó sin break.

```
for numero in numeros:  
    if numero == buscado:  
        print("Encontrado")  
        break  
else:  
    print("No encontrado")
```

11. Desempaquetado y asignaciones útiles

Desempaquetado simple

```
a, b = (10, 20)
```

Intercambiar valores

```
a, b = b, a
```

Capturar elementos intermedios

```
primero, *medio, ultimo = [1, 2, 3, 4, 5]
```

- primero = 1

- `medio` = [2, 3, 4]
- `ultimo` = 5

Retornos múltiples

```
def dividir(a, b):
    return a // b, a % b

cociente, residuo = dividir(17, 5)
```

Realmente se retorna una tupla.

12. Colecciones: listas, tuplas, conjuntos y `range`

Comparación general

Tipo	Orden	Duplicados	Mutable	Índice
<code>list</code>	Sí	Sí	Sí	Sí
<code>tuple</code>	Sí	Sí	No	Sí
<code>set</code>	No orden indexable contractual	No	Sí	No
<code>frozenset</code>	No orden indexable contractual	No	No	No
<code>range</code>	Sí	Según secuencia	No	Sí

Listas

```
nombres = ["Ana", "Luis", "Marta"]
```

Métodos frecuentes:

Método	Efecto
<code>append(x)</code>	Agrega al final.
<code>extend(it)</code>	Agrega todos los elementos.
<code>insert(i, x)</code>	Inserta en posición.
<code>remove(x)</code>	Elimina primera ocurrencia.
<code>pop(i=-1)</code>	Elimina y retorna.
<code>index(x)</code>	Busca posición.
<code>count(x)</code>	Cuenta apariciones.
<code>sort()</code>	Ordena in-place.
<code>reverse()</code>	Invierte in-place.
<code>copy()</code>	Copia superficial.

Tuplas

```
coordenada = (6.24, -75.58)
```

Útiles cuando la estructura es fija.

Sets

```
unicos = {1, 2, 3}
```

Útiles para:

- eliminar duplicados;
- comprobar pertenencia de forma eficiente;
- operaciones de conjuntos.

Operaciones de conjuntos

Operación	Ejemplo
Unión	<code>a b</code>
Intersección	<code>a & b</code>
Diferencia	<code>a - b</code>

13. Listas por dentro: mutación, inserciones y costo conceptual

Esta sección no pretende convertir el manual en documentación de implementación interna, pero sí dar el **modelo mental correcto**.

Modelo mental de una lista en CPython

Una lista se comporta conceptualmente como un arreglo dinámico de referencias:

- tiene elementos visibles;
- suele reservar algo de capacidad adicional;
- puede crecer sin tener que reconstruirse en cada `append()`;
- a veces, al crecer mucho, necesita redimensionar su almacenamiento interno.

Por qué `append()` suele ser barato

```
lista.append(x)
```

Si existe capacidad libre al final, se coloca el nuevo elemento y se actualiza el tamaño.

Por qué insertar al inicio cuesta más

```
lista.insert(0, x)
```

Hay que mover hacia la derecha todos los elementos existentes.

Por qué eliminar por posición intermedia cuesta más

```
del lista[i]
```

Después de quitar el elemento, los posteriores deben correrse hacia la izquierda.

Relación con ejercicios manuales

Cuando implementas funciones propias tipo:

- `insert` desplazando a la derecha;
- `deletebyposition` desplazando a la izquierda;
- `count` recorriendo elementos;

estás reproduciendo, a nivel lógico, parte de lo que hacen operaciones nativas por dentro.

Métodos que modifican y retornan `None`

```
lista = [3, 1, 2]
resultado = lista.sort()
```

- `lista` queda ordenada;
- `resultado` es `None`.

Por eso esto es un error común:

```
lista = lista.sort() # lista pasa a ser None
```

14. Diccionarios y mapeos

Un diccionario asocia claves con valores.

```
estudiante = {
    "nombre": "Ana",
    "nota": 4.5
}
```

Propiedades

Concepto	Explicación
Clave	Debe ser hashable.
Valor	Puede ser cualquier objeto.
Acceso	<code>dic[clave]</code> .
Orden	Conserva inserción en Python moderno.

Métodos frecuentes

Método	Uso
<code>get(k, default=None)</code>	Acceso seguro.
<code>update(...)</code>	Agrega o sobrescribe.
<code>pop(k)</code>	Elimina y retorna valor.
<code>keys()</code>	Vista de claves.
<code>values()</code>	Vista de valores.
<code>items()</code>	Vista de pares.

`dic[clave]` VS `get()`

```
dic["edad"]
```

lanza `KeyError` si falta.

```
dic.get("edad", 0)
```

retorna `0` si falta.

15. Mutabilidad, aliasing y copias

Tipos mutables e inmutables

Inmutables	Mutables
<code>int</code> , <code>float</code> , <code>bool</code> , <code>str</code> , <code>tuple</code> , <code>frozenset</code>	<code>list</code> , <code>dict</code> , <code>set</code> , <code>bytearray</code>

Alias

```
a = [1, 2]
b = a
```

`a` y `b` son alias del mismo objeto.

Copia superficial

```
b = a.copy()  
# o  
b = a[:]
```

Copia la lista externa, pero si dentro hay listas, esas internas pueden seguir compartidas.

Copia profunda

```
import copy  
b = copy.deepcopy(a)
```

Copia recursivamente.

Ejemplo con listas anidadas

```
matriz = [[1, 2], [3, 4]]  
copia = matriz.copy()  
copia[0][0] = 99
```

También cambia `matriz[0][0]`, porque la sublista interna se compartió.

16. Comprensiones, iterables, iteradores y generadores

Comprensión de listas

```
cuadrados = [n ** 2 for n in range(10)]
```

Con condición

```
pares = [n for n in range(20) if n % 2 == 0]
```

Diccionarios por comprensión

```
longitudes = {palabra: len(palabra) for palabra in palabras}
```

Iterable vs iterador

Término	Idea
Iterable	Algo que puede recorrerse.

Término	Idea
Iterador	Objeto que entrega elementos uno por uno y se agota.

```
iterador = iter([10, 20, 30])
next(iterador) # 10
```

Generadores

```
def cuenta_regresiva(n):
    while n > 0:
        yield n
        n -= 1
```

No construyen toda la lista en memoria de una vez; producen valores cuando se solicitan.

17. Funciones, parámetros, valores por defecto y `return`

Definición general

```
def promedio(valores):
    return sum(valores) / len(valores)
```

Parámetros y argumentos

Concepto	Significado
Parámetro	Nombre en la definición de la función.
Argumento	Valor entregado al llamarla.

Argumentos posicionales y por palabra clave

```
def saludar(nombre, tono="formal"):
    ...

saludar("Ana")
saludar(nombre="Ana", tono="cercano")
```

Valores por defecto

```
def imprimir_tabla(tabla, ancho, encabezado=None):
    ...
```

Esto permite llamar de dos formas:

```
imprimir_tabla(datos, 12)
imprimir_tabla(datos, 12, ["Nombre", "Nota"])
```

return

Caso	Resultado
return valor	Termina y entrega valor.
return	Termina y retorna <code>None</code> .
Sin return	Retorna <code>None</code> implícito.

Efectos secundarios

Una función puede:

- retornar un valor;
- imprimir;
- escribir un archivo;
- modificar una lista recibida.

Conviene distinguir esos casos.

*args y **kwargs

```
def sumar(*numeros):
    return sum(numeros)
```

*args agrupa posicionales extra en una tupla.

```
def mostrar(**datos):
    print(datos)
```

**kwargs agrupa nombrados extra en un diccionario.

18. Scope, namespaces, LEGB, global y nonlocal

Scope

El scope determina desde dónde puede verse un nombre.

Regla LEGB

Nivel	Significado
L	Local.
E	Enclosed, función externa envolvente.
G	Global del módulo.

Nivel	Significado
B	Built-ins.

```
x = 10

def f():
    x = 5
    print(x)

f()      # 5
print(x) # 10
```

global

```
contador = 0

def incrementar():
    global contador
    contador += 1
```

Úsalo con moderación.

nonlocal

Sirve en funciones anidadas cuando quieres modificar un nombre de la función externa.

19. Recursión, funciones anidadas y cierres básicos

Recursión

Una función recursiva se llama a sí misma.

```
def factorial(n):
    if n <= 1:
        return 1
    return n * factorial(n - 1)
```

Necesita:

- caso base;
- avance hacia ese caso.

Funciones anidadas

```
def exterior():  
    mensaje = "Hola"  
  
    def interior():  
        print(mensaje)  
  
    return interior
```

Closure

La función interna puede recordar variables de la externa, incluso después de que la externa haya terminado.

20. Módulos, imports, paquetes y nombres de archivo

Módulos

Un archivo `.py` importable funciona como módulo.

```
import math  
print(math.sqrt(9))
```

Formas de importar

Forma	Comentario
<code>import modulo</code>	Clara y explícita.
<code>from modulo import nombre</code>	Útil si usarás mucho ese nombre.
<code>import modulo as alias</code>	Conveniente para alias comunes.
<code>from modulo import *</code>	Evítalo en proyectos mantenibles.

Paquetes

Un paquete organiza módulos en carpetas.

```
proyecto/  
├─ main.py  
├─ funciones.py  
└─ utilidades.py
```

Nombres de archivos

Prefiere:

```
funciones.py
pruebas_caja_negra.py
sistema_notas.py
```

Evita espacios y tildes en nombres de archivos para simplificar imports y comandos.

21. `main()`, `__name__ == "__main__"` y estructura de programas

Script pequeño

En un ejercicio muy corto, esto es suficiente:

```
nombre = input("Nombre: ")
print("Hola", nombre)
```

Programa más grande

En un proyecto con menú, archivos y funciones, suele ser más claro:

```
def main():
    # flujo principal
    ...

if __name__ == "__main__":
    main()
```

¿Qué es `__name__`?

Python asigna un nombre especial a cada módulo.

- Si ejecutas el archivo directamente, `__name__` vale `"__main__"`.
- Si el archivo es importado desde otro archivo, `__name__` toma el nombre del módulo.

¿Por qué usar la condición?

Evita que el programa se ejecute automáticamente cuando importas sus funciones desde otro archivo.

Sin protección

```
# main.py
print("Se ejecuta de inmediato")
```

Si otro archivo hace:

```
import main
```

se imprimirá ese mensaje.

Con protección

```
def main():
    print("Solo si ejecuto este archivo")

if __name__ == "__main__":
    main()
```

Al importar, no se dispara el programa principal.

¿Se ve “demasiado formal”? Depende

Caso	Recomendación
Ejercicio de 10 líneas	Puede no hacer falta.
Script personal simple	Opcional.
Proyecto con varios módulos	Recomendado.
Archivo pensado para importar funciones	Muy recomendable.

Idea clave: no es “escribir como IA”; es separar el flujo principal del código reutilizable. Se usa cuando el proyecto ya lo justifica.

22. Archivos, rutas, `__file__`, `os.path` y `pathlib`

Abrir archivos

```
with open("datos.txt", "r", encoding="utf-8") as archivo:
    contenido = archivo.read()
```

`with` garantiza cierre correcto aunque ocurra un error.

Modos de apertura

Modo	Significado
<code>r</code>	Leer.
<code>w</code>	Escribir reemplazando contenido.
<code>a</code>	Agregar al final.
<code>x</code>	Crear solo si no existe.
<code>b</code>	Binario.
<code>+</code>	Lectura y escritura combinadas.

Leer archivos

Método	Uso
<code>read()</code>	Todo el archivo.
<code>readline()</code>	Una línea.
<code>readlines()</code>	Lista de líneas.
<code>for linea in archivo:</code>	Recorrido línea por línea.

Rutas relativas y absolutas

Tipo	Ejemplo	Significado
Relativa	<code>"datos.txt"</code>	Depende del directorio actual.
Absoluta	<code>"C:/proyecto/datos.txt"</code>	Ubicación completa.

Problema típico

Si ejecutas desde VS Code o Spyder, el **directorio actual** puede no ser la carpeta del archivo `.py`. Entonces esto puede fallar:

```
ARCHIVO = "registros.txt"
```

Solución con `__file__` y `os.path`

```
import os

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
ARCHIVO = os.path.join(BASE_DIR, "registros.txt")
```

Desarmado línea por línea

Fragmento	Qué significa
<code>__file__</code>	Ruta del archivo <code>.py</code> actual.
<code>os.path.abspath(__file__)</code>	Convierte esa ruta en absoluta.
<code>os.path.dirname(...)</code>	Toma la carpeta contenedora.
<code>os.path.join(BASE_DIR, "registros.txt")</code>	Une carpeta + nombre de archivo de forma compatible con el sistema operativo.

Versión con `pathlib`

```
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent
ARCHIVO = BASE_DIR / "registros.txt"
```

`pathlib` suele ser más expresivo en proyectos modernos.

23. Excepciones, validación de input y errores controlados

Excepciones comunes

Excepción	Ejemplo
ValueError	<code>int("hola")</code>
TypeError	<code>"1" + 2</code>
ZeroDivisionError	<code>10 / 0</code>
IndexError	<code>lista[99]</code>
KeyError	<code>dic["x"]</code> si falta
FileNotFoundError	Abrir archivo inexistente

try / except

```
while True:
    try:
        edad = int(input("Edad: "))
        break
    except ValueError:
        print("Ingrese un número entero válido")
```

else y finally

```
try:
    x = int(texto)
except ValueError:
    print("No válido")
else:
    print("Conversión correcta")
finally:
    print("Este bloque siempre se ejecuta")
```

raise

```
if nota < 0:
    raise ValueError("La nota no puede ser negativa")
```

Validar temprano

Buena práctica:

- validar documento antes de usarlo;
- validar rango antes de guardar;

- validar que una opción exista antes de indexar una lista.

24. Programación orientada a objetos

Clase e instancia

```
class Estudiante:
    def __init__(self, nombre, nota):
        self.nombre = nombre
        self.nota = nota
```

```
ana = Estudiante("Ana", 4.5)
```

self

`self` es una convención para el primer parámetro de métodos de instancia. No es palabra reservada, pero debe usarse por claridad.

Atributos de instancia y de clase

```
class Producto:
    iva = 0.19 # atributo de clase

    def __init__(self, precio):
        self.precio = precio # atributo de instancia
```

Métodos

```
def total(self):
    return self.precio * (1 + self.iva)
```

Herencia

```
class Animal:
    def hablar(self):
        print("...")

class Perro(Animal):
    def hablar(self):
        print("Guau")
```

super()

`super()` continúa la búsqueda de métodos según el MRO, no significa siempre “el padre inmediato” en diseños complejos.

Métodos especiales frecuentes

Método	Se usa en
<code>__str__</code>	<code>print(obj)</code>
<code>__repr__</code>	Depuración.
<code>__len__</code>	<code>len(obj)</code>
<code>__eq__</code>	<code>==</code>
<code>__lt__</code>	<code><</code>
<code>__iter__</code>	Iteración.
<code>__getitem__</code>	Corchetes.

Decoradores de clase

Decorador	Uso
<code>@staticmethod</code>	Función organizada dentro de la clase, sin <code>self</code> ni <code>cls</code> .
<code>@classmethod</code>	Método que recibe <code>cls</code> .
<code>@property</code>	Expone método como atributo.

25. Anotaciones de tipo, `typing` y `dataclasses`

Type hints

```
def promedio(notas: list[float]) -> float:
    return sum(notas) / len(notas)
```

Las anotaciones ayudan a:

- lectores;
- IDEs;
- linters;
- verificadores de tipos.

Python no las hace cumplir automáticamente en tiempo de ejecución.

Tipos compuestos

```
nombre: str
notas: list[float]
matriz: list[list[float]]
config: dict[str, int]
```

Unión de tipos

```
def buscar(nombre: str) -> str | None:
    ...
```

Significa: retorna `str` o `None`.

dataclasses

```
from dataclasses import dataclass

@dataclass
class Producto:
    nombre: str
    precio: float
    cantidad: int = 0
```

`@dataclass` puede generar automáticamente métodos como `__init__` y `__repr__`.

26. Biblioteca estándar de consulta frecuente

Mapa rápido

Módulo	Para qué sirve
<code>math</code>	Matemática real.
<code>random</code>	Aleatoriedad.
<code>statistics</code>	Estadística básica.
<code>datetime</code>	Fechas y horas.
<code>time</code>	Pausas y medición temporal.
<code>json</code>	Leer y escribir JSON.
<code>csv</code>	Archivos CSV.
<code>os</code>	Sistema operativo.
<code>sys</code>	Interacción con intérprete.
<code>pathlib</code>	Rutas.
<code>copy</code>	Copias profundas/superficiales.
<code>re</code>	Expresiones regulares.
<code>logging</code>	Registro de eventos.
<code>argparse</code>	Argumentos de línea de comandos.
<code>unittest</code>	Pruebas automáticas.
<code>venv</code>	Entornos virtuales.

Módulo	Para qué sirve
<code>collections</code>	Estructuras especializadas.
<code>itertools</code>	Iteradores eficientes.
<code>functools</code>	Herramientas funcionales.

json

```
import json

datos = {"nombre": "Ana", "nota": 4.5}
texto = json.dumps(datos, ensure_ascii=False, indent=2)
```

csv

```
import csv

with open("datos.csv", newline="", encoding="utf-8") as archivo:
    lector = csv.reader(archivo)
    for fila in lector:
        print(fila)
```

datetime

```
from datetime import datetime

ahora = datetime.now()
texto = ahora.strftime("%Y-%m-%d %H:%M:%S")
```

time.perf_counter()

Útil para medir duración de código.

venv

```
python -m venv .venv
```

Un entorno virtual separa dependencias de cada proyecto. Normalmente no se sube a Git.

27. Expresiones regulares y strings crudos

Las expresiones regulares permiten buscar patrones en texto.

Módulo `re`

```
import re

patron = r"\d{10}"
texto = "Documento: 1234567890"
coincidencia = re.search(patron, texto)
```

¿Por qué `r"..."`?

Una string cruda evita que Python interprete escapes como `\n` o `\t` antes de que la expresión regular los procese.

```
r"\n"  # barra + n
"\n"   # salto de línea real
```

Funciones frecuentes

Función	Uso
<code>re.search()</code>	Busca en cualquier parte.
<code>re.match()</code>	Busca al inicio.
<code>re.fullmatch()</code>	Exige coincidencia completa.
<code>re.findall()</code>	Lista de coincidencias.
<code>re.sub()</code>	Sustituye patrones.

28. Depuración, pruebas y logging

Depurar con `print()`

Es válido en ejercicios y pruebas rápidas:

```
print("valor actual:", valor)
```

`assert`

```
assert validar_documento("1234567890") is True
```

Se usa para verificar condiciones que deberían cumplirse.

Pruebas de caja negra

Se prueban entradas y salidas esperadas sin depender de cómo está implementada internamente la función.

Entrada	Salida esperada
Documento de 10 dígitos	True
Documento corto	False

unittest

La biblioteca estándar incluye un marco formal de pruebas.

logging

Para proyectos más grandes, es mejor que llenar el código de `print()` de depuración.

```
import logging

logging.basicConfig(level=logging.INFO)
logging.info("Programa iniciado")
```

29. Patrones de proyectos: menús, archivos de texto y reportes

Menú principal

```
def menu():
    print("1) Cargar")
    print("2) Salir")
```

Bucle de interacción

```
while True:
    menu()
    opcion = input("Seleccione: ")

    if opcion == "1":
        ...
    elif opcion == "2":
        break
    else:
        print("Opción inválida")
```

Separar flujo y lógica

Archivo	Responsabilidad
main.py	Menú y flujo.
funciones.py	Procesamiento.
utilidades.py	Apoyos generales.

Archivo	Responsabilidad
.txt o .json	Datos persistentes.

Patrón de carga de archivo

1. leer texto;
2. limpiar líneas;
3. dividir por separadores;
4. convertir tipos;
5. validar consistencia.

Serializar datos

Serializar es convertir estructuras a texto o bytes para guardarlas.

Reportes en consola

Para mostrar datos ordenados:

- encabezado;
- filas uniformes;
- formato numérico;
- control de anchos.

30. Eficiencia, búsqueda y ordenamiento

Eficiencia intuitiva

No todos los bucles cuestan lo mismo.

Patrón	Idea
Un bucle simple	Crece proporcionalmente al tamaño.
Dos bucles anidados	Puede crecer mucho más rápido.
Buscar en lista	Recorre potencialmente varios elementos.
Buscar en set/dict	Suele ser más directo para pertenencia.

Ordenamiento burbuja

- compara vecinos;
- intercambia si están en orden incorrecto;
- repite varias pasadas.

Útil para aprender, no suele ser la mejor opción en producción.

Ordenamiento por selección

- busca el mayor o menor;

- lo coloca en su posición;
- repite con el resto.

`sorted()` y `.sort()`

```
nueva = sorted(lista)
lista.sort()
```

Forma	Efecto
<code>sorted(lista)</code>	Crea nueva lista ordenada.
<code>lista.sort()</code>	Modifica la lista original.

Medir tiempo

```
from time import perf_counter

inicio = perf_counter()
# código
fin = perf_counter()
print(fin - inicio)
```

31. Buenas prácticas y trampas frecuentes

Buenas prácticas

Situación	Recomendación
Comparar con <code>None</code>	<code>is None</code> / <code>is not None</code> .
Archivos de texto	<code>with open(..., encoding="utf-8")</code> .
Menús grandes	<code>main()</code> + funciones auxiliares.
Nombres de archivos	<code>snake_case.py</code> .
Imports	Preferir explícitos.
Validación	Antes de usar el dato.
Rutas	Construir desde <code>__file__</code> o <code>pathlib</code> .
Ordenar	Diferenciar <code>sort()</code> y <code>sorted()</code> .

Trampas frecuentes

Error	Qué ocurre
<code>lista = lista.sort()</code>	<code>lista</code> termina siendo <code>None</code> .
Usar <code>== None</code>	Funciona a veces, pero no es la convención correcta.

Error	Qué ocurre
Olvidar convertir <code>input()</code>	El dato sigue siendo string.
Dividir por cero	<code>ZeroDivisionError</code> .
Indexar fuera de rango	<code>IndexError</code> .
Usar ruta relativa frágil	El archivo no aparece según dónde ejecutes.
Copiar lista anidada con <code>.copy()</code>	Las sublistas internas pueden seguir compartidas.
Usar defaults mutables	Estado inesperadamente compartido entre llamadas.

32. Glosario técnico

Término	Definición
Abstracción	Simplificación que conserva lo relevante de un problema.
Alias	Segundo nombre que apunta al mismo objeto.
Argumento	Valor entregado a una función al llamarla.
Atributo	Dato o comportamiento accesible con punto desde un objeto o clase.
Byte	Unidad de 8 bits.
Code point	Número abstracto asignado por Unicode.
Codificación	Reglas para convertir texto a bytes y viceversa.
Colección	Estructura que agrupa varios objetos.
Concurrencia	Varias tareas avanzan en periodos superpuestos.
Context manager	Objeto usado con <code>with</code> para administrar recursos.
Dataclass	Clase enriquecida automáticamente mediante <code>@dataclass</code> .
Descriptor de archivo	Recurso del sistema asociado a un archivo abierto.
Duck typing	Importa el comportamiento disponible, no solo la clase nominal.
Efecto secundario	Cambio observable fuera del valor retornado.
Encapsular	Agrupar estado y comportamiento detrás de una interfaz.
Excepción	Evento anómalo de ejecución que puede capturarse.
Float	Número de punto flotante.
Glifo	Forma visual concreta de un carácter.

Término	Definición
Hash	Valor usado para organizar estructuras como dict y set.
Hashable	Objeto apto como clave de dict o elemento de set.
Identidad	Ser exactamente el mismo objeto.
In-place	Operación que modifica el objeto existente.
Immutable	Objeto que no cambia internamente una vez creado.
Iterable	Objeto que puede recorrerse.
Iterador	Objeto que produce elementos uno a uno y se agota.
MRO	Orden de búsqueda de métodos y atributos en herencia.
Mutable	Objeto cuyo contenido puede modificarse.
Namespace	Espacio que asocia nombres con objetos.
Objeto	Entidad con identidad, tipo y valor.
Parámetro	Nombre declarado en una función para recibir argumentos.
Path	Ruta hacia archivo o carpeta.
Proxy	Intermediario que delega operaciones.
Referencia	Vínculo entre un nombre y un objeto.
Scope	Región donde un nombre es visible.
Serializar	Convertir datos a formato almacenable o transmisible.
Slicing	Corte de una secuencia con <code>inicio:fin:paso</code> .
String cruda	Cadena prefijada con <code>r</code> que no interpreta escapes comunes.
Truncar	Cortar la parte decimal sin redondear.
Unicode	Estándar de representación de caracteres.
Wrapper	Capa que envuelve y añade comportamiento.

33. Fuentes oficiales consultadas

Este manual se apoya principalmente en documentación oficial de Python y en materiales oficiales asociados al lenguaje.

Fuente	Temas contrastados
Python Standard Library - Built-in Types	Tipos, colecciones, verdad lógica.

Fuente	Temas contrastados
Python Standard Library - Built-in Functions	<code>print</code> , <code>format</code> , <code>sorted</code> , <code>enumerate</code> , <code>zip</code> , etc.
Python Tutorial - Control Flow	Parámetros por defecto, flujo, funciones.
Python Reference - Expressions	Operadores, pertenencia e identidad.
Python Reference - Compound Statements	<code>with</code> , <code>match</code> , estructuras compuestas.
Python Unicode HOWTO	Unicode, bytes y UTF-8.
Python Library - <code>__main__</code>	Entorno principal y punto de entrada.
Python Library - <code>os.path</code> y <code>pathlib</code>	Rutas y manipulación de paths.
Python Library - <code>typing</code>	Anotaciones de tipo.
Python Library - <code>dataclasses</code>	Clases de datos.
Python Library - <code>venv</code>	Entornos virtuales.
Python Library - <code>unittest</code>	Pruebas unitarias.
Python Library - <code>logging</code>	Registro de eventos.
Python Library - <code>re</code>	Expresiones regulares.
Python Library - <code>json</code> , <code>csv</code> , <code>datetime</code> , <code>time</code> , <code>copy</code>	Trabajo con datos, fechas y copias.

Cierre

Un manual de referencia no reemplaza la documentación oficial. Su propósito es **ordenar lo importante, dar intuiciones correctas y servir como puente entre lo que ves en un proyecto y la explicación técnica de fondo.**

Este documento está diseñado para que, cuando aparezca una línea de código que no entiendas, puedas ubicar su idea central, desmontarla en partes y decidir con criterio si esa forma de escribirla vale la pena en tu propio código.